

IE460 Term Project Report

İsmail Hakkı Bakır

Table of Contents

IE460 Term Project Report	1
1. Introduction	2

2. Data Preprocessing	3
3. Methodology	4
3.1 Data Balancing Techniques Used	4
3.2 Classification Algorithms	5
4. Results & Discussion	6
4.1 Dataset 1: Yeast	6
4.2 Dataset 2: Parkinson	8
4.3 Dataset 3: Heart	9
4.4 Dataset 4: CTO	10
4.5 Dataset 5: BCWD	12
4.6 Visual Comparison of Datasets:	13
5. Conclusion	14

1. Introduction

In real-world scenarios, classification problems involve datasets in which class distributions are imbalanced. Mostly, the majority class dominates the minority class because the minority class is represented by a relatively small number of observations. This imbalance leads to a

challenge for many classification algorithms, particularly those that rely on distance measures (such as Euclidean Distance) or overall error minimization, as the learning process tends to favor the majority class while disregarding the minority class.

In some events, the importance of minority classes is extremely huge. For instance, while in medical diagnosis, the minority class is generally the worst case. However, a misclassification of this kind of minority class can cause death. Consequently, evaluating model performance solely based on overall accuracy can be misleading; we should also conduct studies on the overall performance of the minority class.

The objective of this report is to explain the impact of data balancing techniques and k-nearest neighbor (kNN)-based classifiers on classification performance in imbalanced settings (which are provided with various datasets). Specifically, in this report, a comparison of different scenarios has been applied. The compared scenarios can be stated as: with no balancing to those employing synthetic data generation methods, namely SMOTE and G-SMOTE. In addition, several kNN variants are considered, including standard kNN, distance-weighted (modified) kNN, fuzzy kNN, and radius-based kNN. Rather than focusing exclusively on overall accuracy, the analysis emphasizes class-specific performance, particularly the behavior of models on the minority class, in order to provide a more informative and application-relevant assessment of classifier effectiveness.

2. Data Preprocessing

Five different benchmark datasets exhibit varying degrees of class imbalance and feature characteristics. The datasets used are Yeast, Parkinson, Heart, CTO, and BCWD. These datasets have different characteristics and resulted in different conclusion which will be examined in further sections of this report.

The three steps have been applied consecutively in order to prepare and understand the dataset:

- **Data Normalization:** All feature values were normalized using min-max normalization, scaling each feature to the $[0,1]$ range. This was crucially important for kNN-based classifiers, as distance calculations are directly influenced by the scale of individual features. Without normalization, features with larger values cannot be

compared with features from different scales. Furthermore, the normalization has been applied independently for each feature.

- **Splitting the Train & Test Data:** For each dataset, the division has been made into training and testing sets using an 80/20 ratio. The split was performed with a fixed random seed to ensure reproducibility with varying experiments. Furthermore, all training and parameterization operations were conducted exclusively on the training set. The test set was used only once for the final performance evaluation. This separation ensures the prevention of overfitting or data leakage.
- **Identifying Minority & Majority Classes:** Since all datasets are binary classification problems, the minority and majority classes were determined based on their frequency in the training set. Class labels were not assumed a priori; instead, class counts were computed dynamically after the train–test split. This approach avoids potential bias that could arise if class distributions differ slightly between the full dataset and the training subset. The identification of minority and majority classes plays a central role in subsequent analysis, as performance metrics are reported separately for each class to capture the effects of class imbalance more accurately.

3. Methodology

This section describes the methodology adopted to evaluate the effects of data balancing techniques and kNN-based classifiers on imbalanced datasets. Our analysis covers the effects of balancing techniques for different datasets and classifiers. In this section, these balancing techniques and classifiers will be provided for a better understanding of the comparison that will be provided in the next section.

3.1 Data Balancing Techniques Used

Three balancing strategies were considered, as stated in the instructions of the project. Importantly, all these techniques were applied to training sets for all datasets, both during

cross-validation and final model training. The test sets were left untouched to preserve the integrity of the experiment. These three strategies have been explained below:

1. **No Balancing:** This was the baseline setting for better comparison. In this setting, the original class distribution of the training data was preserved. This configuration serves as a reference point to assess the impact of balancing techniques on classifier behavior.
2. **SMOTE:** The Synthetic Minority Over-sampling Technique (SMOTE) was employed to generate synthetic minority class samples by interpolating between existing minority instances and their nearest neighbors. The number of synthetic samples was determined such that the minority and majority class sizes in the training set became equal.
3. **G-SMOTE:** Instead of relying solely on linear interpolation between minority neighbors as we did with SMOTE, G-SMOTE generates synthetic samples by sampling directions in the feature space around minority class centers and creating new points within a controlled geometric region.

3.2 Classification Algorithms

In this study, four kNN variants were applied separately. All classifiers rely on the normalized datasets and use Euclidean Distance. These four classifiers have been provided below:

1. **Standard kNN:** The standard kNN classifier assigns a class label to a test instance based on the majority vote among its *k* nearest neighbors in the training set. Unfortunately, this approach is particularly sensitive to class imbalance, as majority class neighbors tend to dominate the voting process, and this sensitivity will be analyzed for the specific datasets used in this study in further sections.
2. **Modified kNN :** The modified kNN classifier assigns weights to neighbors based on their distance to the center point, giving stronger influence to closer neighbors.
3. **Fuzzy kNN:** Fuzzy kNN extends the standard approach by assigning class memberships to each instance. Each neighbor contributes to class membership scores according to a distance-based weighting function, and the final class label is determined by the majority.

4. **Radius – based kNN (rNN):** The radius-based kNN classifier assigns a class label based on all training instances that fall within a fixed distance (radius) of the center point.

The selection of k is critical for all these classifiers. To select an appropriate value, a 5-fold cross-validation procedure was applied on the training set. Candidate values of k ranged from 1 to 10.

Furthermore, the performance evaluations of each classifier have been conducted, and in the next section, we will deep-dive into these analyses.

4. Results & Discussion

This section consolidates the results of the study for each dataset. All balancing methods and classifiers have been compared according to their efficiencies. For each dataset, the discussion follows the same structure: baseline behavior without balancing, the effect of SMOTE and G-SMOTE on minority/majority performance, classifier-specific patterns, and the most defensible configuration depending on the objective.

4.1 Dataset 1: Yeast

The comprehensive comparison table has been provided in Table 1. The comparison for each balancing method has been provided below:

1. Baseline:

Under no balancing, Standard/Modified/Fuzzy kNN achieve high overall accuracy (approximately 0.94) with high majority accuracy (approx. 0.97), but minority accuracy remains moderate (approx. 0.69–0.75). However, a different baseline behavior appears in Radius kNN, where overall accuracy is still relatively high (0.90878) because majority accuracy is extremely high, but minority accuracy collapses to 0.15625. This is a clear example of how overall accuracy can tell us nothing about the accuracy of the minority class.

2. Effect of SMOTE:

SMOTE increases minority accuracy across most classifiers, but the trade-off differs by model:

Standard/Modified/Fuzzy show modest minority gains (e.g., Modified reaches **0.81250** minority accuracy) with some majority accuracy reduction.

However, Radius kNN changes character entirely: minority accuracy rises sharply (**0.93750**) while majority accuracy drops (**0.78788**), pulling overall accuracy down (**0.80405**). This indicates that, once local density around minority points is increased, the fixed-radius neighborhood starts capturing minority structure effectively, but the decision boundary becomes less conservative toward the majority class.

3.Effect of G-SMOTE:

G-SMOTE further strengthens minority performance, especially for Radius kNN (minority accuracy **0.96875**) while keeping majority accuracy higher than the SMOTE-radius case (**0.84091 vs 0.78788**). Among the non-radius methods, G-SMOTE + Standard yields a strong minority gain (**0.87500**) while maintaining a reasonable majority accuracy (**0.91667**), producing a more balanced profile than the SMOTE cases.

As a result, if the minority-focused strategy is selected, the best case is G-SMOTE + Radius since its minority accuracy is highest. However, if a balanced profile is selected, G-SMOTE + Standard can be applied.

Dataset	Balancing	Classifier	Best_k	Overall_Acc	Minority_Acc	Majority_Acc
Yeast	None	Standard	9	0.94257	0.68750	0.97348
Yeast	None	Modified	5	0.94595	0.75000	0.96970
Yeast	None	Fuzzy	10	0.93919	0.68750	0.96970
Yeast	None	Radius	1	0.90878	0.15625	100.000
Yeast	SMOTE	Standard	1	0.89865	0.71875	0.92045
Yeast	SMOTE	Modified	3	0.89865	0.81250	0.90909
Yeast	SMOTE	Fuzzy	2	0.90541	0.75000	0.92424
Yeast	SMOTE	Radius	10	0.80405	0.93750	0.78788
Yeast	G-SMOTE	Standard	3	0.91216	0.87500	0.91667
Yeast	G-SMOTE	Modified	2	0.90203	0.78125	0.91667
Yeast	G-SMOTE	Fuzzy	2	0.88176	0.78125	0.89394

Yeast	G-SMOTE	Radius	10	0.85473	0.96875	0.84091
--------------	---------	--------	----	---------	---------	---------

Table 1: The comparison of different balancing methods and classifiers for the Yeast Dataset.

4.2 Dataset 2: Parkinson

The results for each classifier and balancing method have been provided in Table 2 for the Parkinson Dataset. We should point out that the minority accuracy is 1.00 in every scenario, and this has a reason:

This pattern strongly suggests that the feature space already separates classes cleanly for the minority class in the held-out test split. In this setting, balancing does not solve a minority-detection failure; instead, it may reduce majority accuracy and overall accuracy.

1. Baseline:

Standard/Modified/Fuzzy are identical here: overall accuracy **0.97436** with majority accuracy **0.96552** and minority accuracy **1.000**. Radius kNN has lower overall accuracy (**0.92308**) due to reduced majority accuracy (**0.89655**), despite perfect minority accuracy.

2. Effect of SMOTE and G-SMOTE.

Both SMOTE and G-SMOTE generally reduce majority accuracy relative to the baseline, and therefore reduce overall accuracy. The drop is most visible in the modified/fuzzy cases (e.g., overall 0.92308 or 0.89744 depending on the classifier), while the minority remains perfect throughout. This indicates that balancing does not provide additional value for minority detection.

As a result, for this dataset, balancing should be avoided. The Parkinson Dataset functions as a “high-separability” dataset. The results are consistent with a scenario where minority instances are already clustered and easily recoverable with local neighborhood methods after normalization. The appropriate strategy would be Baseline + Standard/Modified/Fuzzy (since they are identical).

Dataset	Balancing	Classifier	Best_k	Overall_Acc	Minority_Acc	Majority_Acc
Parkinson	None	Standard	3	0.97436	100.000	0.96552
Parkinson	None	Modified	3	0.97436	100.000	0.96552

Parkinson	None	Fuzzy	3	0.97436	100.000	0.96552
Parkinson	None	Radius	1	0.92308	100.000	0.89655
Parkinson	SMOTE	Standard	1	0.94872	100.000	0.93103
Parkinson	SMOTE	Modified	4	0.92308	100.000	0.89655
Parkinson	SMOTE	Fuzzy	5	0.89744	100.000	0.86207
Parkinson	SMOTE	Radius	1	0.92308	100.000	0.89655
Parkinson	G-SMOTE	Standard	1	0.94872	100.000	0.93103
Parkinson	G-SMOTE	Modified	4	0.92308	100.000	0.89655
Parkinson	G-SMOTE	Fuzzy	5	0.89744	100.000	0.86207
Parkinson	G-SMOTE	Radius	1	0.92308	100.000	0.89655

Table 2: The comparison of different balancing methods and classifiers for the Parkinson Dataset.

4.3 Dataset 3: Heart

Heart is the most illustrative dataset for the **minority–majority trade-off** because baseline performance is uneven, and balancing changes the minority outcome. The comparisons have been illustrated in Table 3.

1. Baseline:

Overall accuracy is moderate (roughly **0.75–0.85**, depending on the classifier). Majority accuracy is high for Standard (**0.95238**), but minority accuracy is substantially lower (**0.58824**). Modified improves minority accuracy relative to Standard (**0.64706**) but still leaves a gap. Fuzzy performs worst on the minority (**0.47059**) despite high majority accuracy (**0.92857**). The radius baseline is weaker overall (**0.74576**) and does not improve the minority beyond Standard.

2. Effect of SMOTE:

SMOTE increases minority accuracy meaningfully, particularly for Modified and Fuzzy:

- Modified reaches **0.82353 minority accuracy**, but majority accuracy drops to **0.76190**, pulling overall to **0.77966**.
- Fuzzy also reaches **0.82353 minority accuracy**, with a similar majority reduction.

This is an example of the expected trade-off: balancing improves minority sensitivity, but majority correctness declines.

3. Effect of G-SMOTE:

G-SMOTE produces a more mixed result:

- Standard increases minority accuracy to **0.76471** while keeping majority at **0.80952** (**overall 0.79661**), which is a comparatively balanced outcome.
- Modified retains high minority (**0.82353**) but majority declines further (**0.73810**).
- Fuzzy does not match SMOTE on minority (**0.70588**).

Dataset	Balancing	Classifier	Best_k	Overall_Acc	Minority_Acc	Majority_Acc
Heart	None	Standard	10	0.84746	0.58824	0.95238
Heart	None	Modified	4	0.81356	0.64706	0.88095
Heart	None	Fuzzy	9	0.79661	0.47059	0.92857
Heart	None	Radius	1	0.74576	0.58824	0.80952
Heart	SMOTE	Standard	3	0.74576	0.64706	0.78571
Heart	SMOTE	Modified	9	0.77966	0.82353	0.76190
Heart	SMOTE	Fuzzy	8	0.79661	0.82353	0.78571
Heart	SMOTE	Radius	3	0.76271	0.64706	0.80952
Heart	G-SMOTE	Standard	7	0.79661	0.76471	0.80952
Heart	G-SMOTE	Modified	10	0.76271	0.82353	0.73810
Heart	G-SMOTE	Fuzzy	6	0.74576	0.70588	0.76190
Heart	G-SMOTE	Radius	5	0.74576	0.58824	0.80952

Table 3: The comparison of different balancing methods and classifiers for the Heart Dataset.

4.4 Dataset 4: CTO

CTO starts with strong overall accuracy even without balancing, but minority accuracy is only moderate. The value of balancing here is to raise minority accuracy while protecting majority accuracy as much as possible.

1. Baseline:

Overall accuracy is high, and majority accuracy is extremely high. Minority accuracy, however, ranges around 0.52–0.67 depending on the classifier. Furthermore, Modified is weakest on the minority (0.51852) despite excellent majority accuracy.

2. Effect of SMOTE:

SMOTE improves minority accuracy across all classifiers:

- Standard/Modified/Fuzzy hits **0.74074** minority accuracy, with majority accuracy decreasing slightly but staying high (approx. **0.97**).
- Radius kNN stands out with minority accuracy **0.88889**, at the cost of a larger majority drop to **0.93970** and lower overall with **0.93647**.

3. Effect of G-SMOTE:

G-SMOTE shows two distinct behaviors:

- For Standard kNN, it preserves excellent majority accuracy (**0.98995**) while improving the minority to **0.74074** and achieving the highest overall accuracy in the CTO block (**0.97412**). This is a strong “best overall” candidate.
- For Modified, G-SMOTE increases minority further to **0.85185**, with majority at **0.96734 (overall 0.96000)**. This is a better minority-oriented balance than SMOTE + Modified.
- Radius remains minority-strong (**0.88889**) with better majority than SMOTE-radius (**0.95729 vs 0.93970**), making it a robust minority-focused option.

As a result, we have concluded that balancing can produce minority gains without destroying majority performance, particularly with G-SMOTE + Standard, which improves minority detection while leaving majority performance nearly unchanged for the CTO dataset.

Dataset	Balancing	Classifier	Best_k	Overall_Acc	Minority_Acc	Majority_Acc
CTO	None	Standard	1	0.96941	0.66667	0.98995
CTO	None	Modified	3	0.96471	0.51852	0.99497
CTO	None	Fuzzy	4	0.96471	0.55556	0.99246

CTO	None	Radius	1	0.96235	0.55556	0.98995
CTO	SMOTE	Standard	1	0.95529	0.74074	0.96985
CTO	SMOTE	Modified	2	0.96000	0.74074	0.97487
CTO	SMOTE	Fuzzy	2	0.95765	0.74074	0.97236
CTO	SMOTE	Radius	6	0.93647	0.88889	0.93970
CTO	G-SMOTE	Standard	1	0.97412	0.74074	0.98995
CTO	G-SMOTE	Modified	3	0.96000	0.85185	0.96734
CTO	G-SMOTE	Fuzzy	2	0.97176	0.70370	0.98995
CTO	G-SMOTE	Radius	7	0.95294	0.88889	0.95729

Table 4: The comparison of different balancing methods and classifiers for the CTO Dataset.

4.5 Dataset 5: BCWD

On the other hand, the BCWD dataset differs from others. It is high-performing in nearly all settings. The differences between balancing methods and classifiers appear mainly as small shifts rather than extreme changes.

1. Baseline:

Overall accuracy is already high (**up to 0.97345**). Minority accuracy is also high (**0.90476–0.95238, depending on the classifier**). Majority accuracy is consistently very high; in some configurations, it reaches 1.000.

2. Effect of SMOTE and G-SMOTE:

Balancing does not yield dramatic minority gains because there is no large minority failure to correct in the baseline scenario. Instead, balancing tends to slightly reduce majority accuracy in some configurations (e.g., SMOTE + Standard drops majority from **0.98592 to 0.94366**), while keeping minority accuracy near the same level.

A notable case is G-SMOTE + Fuzzy, which returns to the best overall accuracy level (**0.97345**) while maintaining minority at **0.95238** and majority at **0.98592**—essentially matching the strongest baseline behavior.

It has concluded that BCWD behaves like a dataset with a strong class structure under normalization. Here, balancing functions does not have a high necessity. The most defensible

approach is to prefer configurations that preserve the already-strong majority accuracy while maintaining minority performance.

Dataset	Balancing	Classifier	Best_k	Overall_Acc	Minority_Acc	Majority_Acc
BCWD	None	Standard	7	0.97345	0.95238	0.98592
BCWD	None	Modified	4	0.97345	0.92857	100.000
BCWD	None	Fuzzy	9	0.96460	0.92857	0.98592
BCWD	None	Radius	1	0.95575	0.90476	0.98592
BCWD	SMOTE	Standard	7	0.94690	0.95238	0.94366
BCWD	SMOTE	Modified	4	0.95575	0.92857	0.97183
BCWD	SMOTE	Fuzzy	9	0.94690	0.95238	0.94366
BCWD	SMOTE	Radius	7	0.92920	0.90476	0.94366
BCWD	G-SMOTE	Standard	9	0.96460	0.95238	0.97183
BCWD	G-SMOTE	Modified	10	0.94690	0.95238	0.94366
BCWD	G-SMOTE	Fuzzy	10	0.97345	0.95238	0.98592
BCWD	G-SMOTE	Radius	8	0.95575	0.90476	0.98592

Table 5: The comparison of different balancing methods and classifiers for the BCWD Dataset.

4.6 Visual Comparison of Datasets:

Across the five datasets, the accuracy trade-off plots provided in Figure 1 demonstrate a consistent pattern in how balancing methods shift classifier behavior with varying scenarios. For Yeast, Heart, and CTO, moving from no balancing to SMOTE and then to G-SMOTE produces a clear increase in minority accuracy, while majority accuracy declines to varying degrees, which does not conflict with the fundamental trade-off behind balancing. This effect is strongest in Yeast and CTO, where minority accuracy rises extremely, indicating that balancing substantially improves local neighborhood density for the minority class. In contrast, Heart shows a more moderate minority gain accompanied by a noticeable majority drop, reflecting a less clean separation between classes. On the other hand, Parkinson creates an exception: minority accuracy remains at 1.0 across all balancing strategies, while majority accuracy decreases slightly with balancing, which resulted in the dataset being already highly

separable, and that balancing offers little benefit. Finally, BCWD exhibits minimal variation across balancing methods; both minority and majority accuracies remain high and stable, indicating that the dataset is well-structured and largely insensitive to oversampling. So, balancing is not an extreme concern. Overall, the plots confirm that balancing methods are most beneficial in datasets where minority detection is weak at baseline, while in already well-separated datasets, they provide limited gains and may even reduce majority performance in some cases.

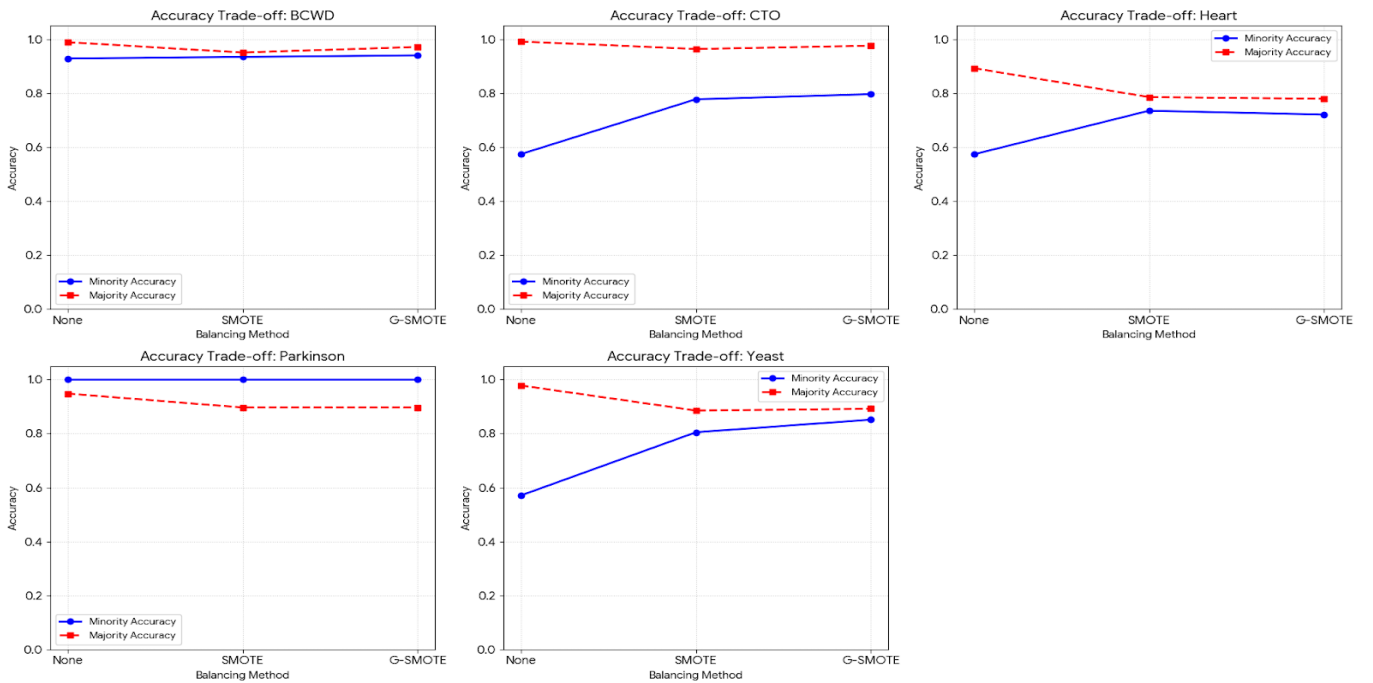


Figure 1: The Accuracy Trade-off Plot for Each Dataset

5. Conclusion

To conclude the study, the results show that balancing methods and classifier applications should be selected appropriately for every case and dataset since each dataset has unique characteristics. Also, balancing methods are most effective when the baseline model struggles to detect the minority class, substantially improving minority accuracy at the cost of some majority performance, and this is the fundamental trade-off in balancing the dataset.

Furthermore, G-SMOTE generally provides a more controlled trade-off than SMOTE, while

datasets with strong inherent separability benefit little from balancing and may even experience unnecessary performance degradation.

6. Appendix

```
%% IE 460
%% İsmail Hakkı Bakır
clc; clear; close all;

% datasets and Parameters
All_datasets = {'Yeast', 'Parkinson', 'Heart', 'CTO', 'BCWD'};
balancing_methods = {'None', 'SMOTE', 'G-SMOTE'};
classifiers = {'Standard', 'Modified', 'Fuzzy', 'Radius'};
k_values = 1:10;

all_results = table();

for d = 1:length(All_datasets)
    dataset = All_datasets{d};
    fprintf('\n===== Veri Seti: %s =====\n', dataset);

    % Data pre-processing
    data_table = readtable([dataset, '.xlsx'], 'Sheet', dataset);
    X1 = table2array(data_table(:, 1:end-1));
    Y1 = table2array(data_table(:, end));

    % Min-Max Norm.
    X_normalized = (X1 - min(X1)) ./ (max(X1) - min(X1) + 1e-10);

    % %80 Train, %20 Test
    rng(42);
    cv_part = cvpartition(Y1, 'HoldOut', 0.20);
    X_train = X_normalized(training(cv_part), :);
    Y_train = Y1(training(cv_part), :);
    X_test = X_normalized(test(cv_part), :);
    Y_test = Y1(test(cv_part), :);

    % Class Identification
    classes = unique(Y_train);
    counts = [sum(Y_train == classes(1)), sum(Y_train == classes(2))];
    [~, min_idx] = min(counts);
    minority_class = classes(min_idx);
    [~, max_idx] = max(counts);
    majority_class = classes(max_idx);

    % Balancing Loop
    for b = 1:length(balancing_methods)
        balancing_type = balancing_methods{b};
```

```

% Classifier Loop
for c = 1:length(classifiers)
    curr_classifier = classifiers{c};
    fprintf('İşleniyor: %s + %s\n', balancing_type, curr_classifier);

    % 5-Fold Cross Validation and choosing k
    cv_kfold = cvpartition(Y_train, 'Kfold', 5);
    k_errors = zeros(length(k_values), 1);

    for k_idx = 1:length(k_values)
        k = k_values(k_idx);
        fold_err = zeros(5, 1);

        for f = 1:5
            X_train_f = X_train(training(cv_kfold, f), :);
            Y_train_f = Y_train(training(cv_kfold, f));
            X_test_f = X_train(test(cv_kfold, f), :);
            Y_test_f = Y_train(test(cv_kfold, f));

            % Balance the Train Data
            if strcmp(balancing_type, 'SMOTE')
                [X_bal, Y_bal] = smote_custom(X_train_f, Y_train_f, 5,
minority_class, majority_class);
            elseif strcmp(balancing_type, 'G-SMOTE')
                [X_bal, Y_bal] = g_smote_custom(X_train_f, Y_train_f,
0.1, minority_class, majority_class);
            else
                X_bal = X_train_f; Y_bal = Y_train_f;
            end

            % Prediction
            preds = run_classifier(X_bal, Y_bal, X_test_f, k,
curr_classifier, classes);
            fold_err(f) = sum(preds ~= Y_test_f) / length(Y_test_f);
        end
        k_errors(k_idx) = mean(fold_err);
    end

    [~, best_idx] = min(k_errors);
    best_k = k_values(best_idx);

    if strcmp(balancing_type, 'SMOTE')
        [X_final_train, Y_final_train] = smote_custom(X_train,
Y_train, 5, minority_class, majority_class);
    elseif strcmp(balancing_type, 'G-SMOTE')
        [X_final_train, Y_final_train] = g_smote_custom(X_train,
Y_train, 0.1, minority_class, majority_class);
    else
        X_final_train = X_train; Y_final_train = Y_train;
    end
end

```

```

        final_preds = run_classifier(X_final_train, Y_final_train, X_test,
best_k, curr_classifier, classes);

        % performance measures
        overall_acc = sum(final_preds == Y_test) / length(Y_test);
        min_mask = (Y_test == minority_class);
        min_acc = sum(final_preds(min_mask) == Y_test(min_mask)) /
sum(min_mask);
        maj_mask = (Y_test == majority_class);
        maj_acc = sum(final_preds(maj_mask) == Y_test(maj_mask)) /
sum(maj_mask);

        res_row = {dataset, balancing_type, curr_classifier, best_k,
overall_acc, min_acc, maj_acc};
        all_results = [all_results; res_row];
    end
end
end

all_results.Properties.VariableNames = {'Dataset', 'Balancing', 'Classifier',
'Best_k', 'Overall_Acc', 'Minority_Acc', 'Majority_Acc'};
disp(all_results);

%% Functions

function p = run_classifier(X_tr, Y_tr, X_te, k, type, classes)
    switch type
        case 'Standard', p = knn_standard(X_tr, Y_tr, X_te, k);
        case 'Modified', p = knn_modified(X_tr, Y_tr, X_te, k, classes);
        case 'Fuzzy',    p = knn_fuzzy(X_tr, Y_tr, X_te, k, 2, classes);
        case 'Radius',   p = knn_radius(X_tr, Y_tr, X_te, 0.3);
    end
end

function [X_balanced, Y_balanced] = smote_custom(X_train, Y_train,
k_neighbors, min_cl, maj_cl)
    X_min = X_train(Y_train == min_cl, :);
    n_min = size(X_min, 1);
    n_maj = sum(Y_train == maj_cl);

    % minority checkk
    if n_min < 2
        X_balanced = X_train; Y_balanced = Y_train; return;
    end

    num_to_generate = n_maj - n_min;
    X_synthetic = zeros(num_to_generate, size(X_train, 2));

```

```

k_safe = min(k_neighbors, n_min - 1);

for i = 1:num_to_generate
    idx = randi(n_min);
    sample = X_min(idx, :);
    dists = sqrt(sum((X_min - sample).^2, 2));
    [~, sorted_idx] = sort(dists);
    neighbor_idx = sorted_idx(randi([2, k_safe + 1]));
    X_synthetic(i, :) = sample + rand() * (X_min(neighbor_idx, :) -
sample);
end
X_balanced = [X_train; X_synthetic];
Y_balanced = [Y_train; repmat(min_cl, num_to_generate, 1)];
end

function [X_balanced, Y_balanced] = g_smote_custom(X_train, Y_train, gamma,
min_cl, maj_cl)
    X_min = X_train(Y_train == min_cl, :);
    n_min = size(X_min, 1);
    if n_min < 1, X_balanced = X_train; Y_balanced = Y_train; return; end

    num_to_generate = sum(Y_train == maj_cl) - n_min;
    X_synthetic = zeros(num_to_generate, size(X_train, 2));

    for i = 1:num_to_generate
        center = X_min(randi(n_min), :);
        direction = randn(1, size(X_train, 2));
        direction = direction / (norm(direction) + 1e-10);
        X_synthetic(i, :) = center + (gamma * rand()) * direction;
    end
    X_balanced = [X_train; X_synthetic];
    Y_balanced = [Y_train; repmat(min_cl, num_to_generate, 1)];
end

function KNN = knn_standard(X_train, Y_train, X_test, k)
    n_test = size(X_test, 1);
    KNN = zeros(n_test, 1);
    for i = 1:n_test
        distances = sqrt(sum((X_train - X_test(i, :)).^2, 2));
        [~, sorted_indices] = sort(distances);
        KNN(i) = mode(Y_train(sorted_indices(1:k)));
    end
end

function m_KNN = knn_modified(X_train, Y_train, X_test, k, classes)
    n_test = size(X_test, 1);
    m_KNN = zeros(n_test, 1);
    for i = 1:n_test
        distances = sqrt(sum((X_train - X_test(i, :)).^2, 2));

```

```

        [sorted_dists, sorted_idx] = sort(distances);
        weights = 1 ./ (sorted_dists(1:k) + 1e-10);
        labels = Y_train(sorted_idx(1:k));
        w1 = sum(weights(labels == classes(1)));
        w2 = sum(weights(labels == classes(2)));
        if w1 >= w2, m_KNN(i) = classes(1); else, m_KNN(i) = classes(2); end
    end
end

function Fuzzy_KNN = knn_fuzzy(X_train, Y_train, X_test, k, m, classes)
    n_test = size(X_test, 1);
    Fuzzy_KNN = zeros(n_test, 1);
    power_val = 2 / (m - 1);
    for i = 1:n_test
        distances = sqrt(sum((X_train - X_test(i, :)).^2, 2));
        [sorted_dists, sorted_idx] = sort(distances);
        weights = 1 ./ (sorted_dists(1:k).^power_val + 1e-10);
        labels = Y_train(sorted_idx(1:k));
        m1 = sum(weights(labels == classes(1)));
        m2 = sum(weights(labels == classes(2)));
        if m1 >= m2, Fuzzy_KNN(i) = classes(1); else, Fuzzy_KNN(i) =
classes(2); end
    end
end

function r_KNN = knn_radius(X_train, Y_train, X_test, gamma)
    n_test = size(X_test, 1);
    r_KNN = zeros(n_test, 1);
    for i = 1:n_test
        distances = sqrt(sum((X_train - X_test(i, :)).^2, 2));
        idx = find(distances <= gamma);
        if isempty(idx), [~, nearest] = min(distances); r_KNN(i) =
Y_train(nearest);
        else, r_KNN(i) = mode(Y_train(idx)); end
    end
end

```